

5. CONFIGURATION DU SITE EN POO

5.1. Utilisation d'une classe

Lorsqu'on conçoit une application, il est recommandé de centraliser les paramètres de configuration dans un fichier : c'est ce qui a été fait dans le fichier `config/config.php`.

- Renommez le fichier `config.php` en `config.class.php`. Définissez une classe `Config` dans ce fichier et ajoutez-y les propriétés requises en les initialisant avec les valeurs permettant de faire fonctionner le site.
- Instanciez l'objet `$Conf` à partir de cette classe dans le fichier `index.php`, avant le routeur, et faites le nécessaire pour que le site fonctionne à nouveau.
- Indication : L'objet `$Conf` est une **variable globale** de l'application.
 - En PHP, une variable globale n'est pas visible dans une classe. Pour rendre une variable globale accessible dans une méthode, il faut la déclarer en tant que tel, au début de cette méthode, avec l'instruction : `global $Conf;`

5.2. Déclaration de valeurs constantes dans une classe

La déclaration d'une valeur constante dans une classe se fait avec le préfixe `const`.

Exemple :

```
public const CONSTANTE = "abc";
```

Par convention, le nom d'une constante s'écrit en caractères majuscules.

Par défaut, la portée d'une constante est `public`, mais on peut lui affecter `protected` ou `private`, comme c'est le cas pour n'importe quelle propriété ou méthode.

Un accès direct à un élément constant (ou statique : voir chapitre suivant) d'une classe peut se faire avec l'emploi de l'opérateur de "résolution de portée" noté `::`.

Cet opérateur impose de désigner la classe dans laquelle l'élément est accédé.

Syntaxe : `classe::élément`

Plusieurs cas peuvent se présenter :

- `maClasse::CONSTANTE` : Accès direct à l'élément `CONSTANTE` déclaré dans la classe `maClasse`.
 - Dans ce cas, il faut que l'élément soit `public`.
- `parent::CONSTANTE` : Accès à l'élément `CONSTANTE` définie dans la classe parente (dans le cas d'un héritage ou extension de classe).
 - L'élément doit être `public` ou `protected`.
- `self::CONSTANTE` : Accès à l'élément `CONSTANTE` déclaré dans la classe où on se trouve.
 - L'élément peut être `public`, `protected` ou `private`.

A noter : Les valeurs constantes d'une classe ne sont pas instanciées pour chaque objet (une constante est commune à tous les objets issus de la classe).

5.3. Les propriétés et méthodes statiques

Une propriété ou une méthode statique appartient à la classe et non à l'objet instancié à partir de cette classe :

- Une propriété statique aura la même valeur pour tous les objets issus de cette classe.
- Une méthode statique aura le même comportement dans tous les objets issus de cette classe.

La déclaration d'une propriété ou d'une méthode statique se fait avec le préfixe `static`.

Exemples :

- Déclaration d'une propriété statique : `public static $maPropriété;`
- Déclaration d'une méthode statique : `public static function maMéthode() { }`

L'accès à une propriété ou à une méthode statique suit le même principe que celui d'une constante :

- `maClasse::$maPropriété` : Accès à une propriété statique déclarée dans la classe `maClasse`.
→ Il faut que la propriété soit `public`.
- `parent::maMéthode()` : Accès à une méthode statique définie dans la classe parente.
→ La méthode doit être `public` ou `protected`.
- `self::$maPropriété` : Accès à une propriété statique déclarée dans la classe où on se trouve.
→ La propriété peut être `public`, `protected` ou `private`.

Remarques :

- Une propriété statique déclarée dans une classe ne pourra pas être modifiée à partir d'un objet instancié à partir de cette classe, mais uniquement à partir des méthodes de cette classe.
- Les propriétés et méthodes statiques d'une classe peuvent être utilisées directement, **sans qu'un objet de cette classe ne soit instancié**.
- L'opérateur habituel `self->maPropriété` ou `$this->maPropriété` n'est pas autorisé pour un élément statique. Seul l'opérateur de "résolution de portée" `::` est autorisé.

Différences en `$this` et `self` :

- `$this` désigne l'objet courant, instancié à partir d'une classe, ou la classe courante. Les propriétés et méthodes "dynamiques" sont accédées avec l'opérateur `->`.
- `self` permet d'accéder aux éléments statiques définis dans une classe avec l'opérateur `::`.

5.4. Configuration par défaut – Configuration personnalisée

Afin d'améliorer la conception du site Magasin, il est utile de prévoir une configuration "par défaut", avec des paramètres censés faire fonctionner le site dans la plupart des cas.

Cette configuration par défaut devra être personnalisée avec les paramètres spécifiques à l'installation du site pour que celui-ci fonctionne correctement.

Principe :

La mise en œuvre repose sur l'utilisation de deux fichiers :

- `includes/default_config.php` : Initialise les paramètres de configuration par défaut (paramètres censés faire fonctionner le site dans la plupart des cas) dans l'objet `$Conf`.
- `config/config.class.php` : Contient la définition des paramètres de configuration personnalisés dans la classe `Config`.

Le fichier `includes/default_config.php` contient les paramètres par défaut, **remplacés si nécessaire** par les paramètres personnalisés déclarés dans `config/config.class.php`, avant d'être appliqués au site :

→ Si un paramètre est défini dans `config.class.php`, il remplace la valeur par défaut initialisée dans `default_config.php`.

Application :

Préparation du fichier de **configuration personnalisée** (`config.class.php`) :

- Classez les paramètres de configuration de la classe `Config` en 3 catégories :
 1. Paramètres qui doivent être fixés en fonction de l'environnement informatique, afin de garantir le bon fonctionnement de l'application.
→ Définissez ces paramètres en tant que propriétés publiques statiques (`static`).
 2. Paramètres constants qui peuvent être personnalisés librement.
→ Définissez ces paramètres en tant que constantes (`const`) publiques.
 3. Paramètres constants qui ne peuvent pas être modifiés sans compromettre le fonctionnement de l'application.
→ Mettez ces paramètres en commentaire (nous y reviendrons plus loin).

Préparation du fichier de **configuration par défaut** :

- Ajoutez le dossier `includes/` sur votre site, et placez-y le nouveau fichier `default_config.php`.
- Dans le fichier `default_config.php`, commencez par inclure le fichier `config.class.php`.

- Initialisez un objet `$Conf` vide dans le fichier avec l'instruction :
`$Conf = new stdClass(); // Objet vide`
→ Dans `index.php`, supprimez l'instanciation de l'objet `$Conf`, ainsi que l'inclusion du fichier `config/config.class.php` qui n'ont plus lieu d'être.
- Initialisez les propriétés de l'objet `$Conf` (vide pour le moment) avec les paramètres de configuration, selon leur catégorie :
 1. Paramètres qui doivent être fixés en fonction de l'environnement informatique, afin de garantir le bon fonctionnement de l'application.
→ Définissez ces paramètres avec leur **valeur par défaut** en utilisant "l'opérateur de fusion **null**" afin de récupérer la **valeur personnalisée**, si cette dernière existe.
Exemple : Initialisation du paramètre `DBHost` (nom du serveur de la BDD) avec :
`$Conf->DBHost = Config::$DB_host ?? "localhost";`
→ Faites une recherche au sujet de l'opérateur de "fusion **null**", encore appelé "null coalescent".
 2. Paramètres constants qui peuvent être personnalisés librement.
→ Définissez ces paramètres en accédant directement à la valeur définie dans la classe `Config`.
 3. Paramètres constants qui ne peuvent pas être modifiés sans compromettre le fonctionnement de l'application.
→ Définissez ces paramètres à partir des valeurs mises en commentaire dans le fichier `config.class.php`.
- Enfin, retirez de la classe `Config` (ou mettez en commentaire) les propriétés dont la valeur par défaut (initialisée par `default_config.php`) est correcte. Retirez également les paramètres mis en commentaire.

Les paramètres de configurations sont maintenant initialisés dans l'objet `$Conf` défini dans le fichier `default_config.php` (et non plus dans le fichier `config.class.php`).

- Faites les modifications requises pour que le site fonctionne correctement
→ Ne modifiez que les fichiers dans lesquels les paramètres de configuration sont requis.

Remarques :

- Vous constaterez que la classe `Config` n'est plus instanciée. De ce fait, il est possible de la rendre abstraite.
→ Modifiez la déclaration de cette classe avec `abstract class Config`.
- L'opérateur **null coalescent** ne fonctionne pas avec des valeurs constantes (ce qui est cohérent).